

信息与软件工程学院

编译技术实验

主讲教师：陈安龙

实验二: LR语法分析

- 请根据LR语法分析实验中给定的文法，利用bison设计语法分析器，该分析器从标准输入读入源代码后，得到该源码对应的抽象语法树。
- 请在页面上传本实验所有源代码及实验报告（源代码至少包括一个flex源文件，即 .l 文件，一个 bison 源文件，即 .y 文件，如有其它 .c 或 .h 文件，也请一并上传）
- 本实验中所构建的抽象语法树结点类型由 nodeType 标识，对该变量的定义包含在 node_type.h 中，此头文件可直接使用，无需上传。

关于Bison/Yacc

① Yacc (Yet Another Compiler Compiler)是生成语法分析器的工具

② Bison 是 GNU 版本的 Yacc

③ Bison会自动生成一个"语法分析器"

关于Bison/Yacc

源程序

a = b + c * d

词法分析器

Flex

词法规则
（正则表达式）

词法记号序列

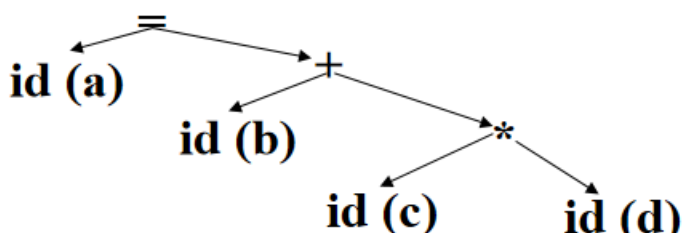
id = id + id * id

语法分析器

Bison

语法规则
（产生式文法）

抽象语法树



代码生成

关于Bison/Yacc

① 词法分析器与语法分析器的交互

flex: yylex()

Bison: yyparser()

② 语法分析器yyparser需要下一个新单词时，调用词法分析子程序yylex，yylex从输入串中识别一个单词后返回。

关于Bison/Yacc

① 既然yyparser要处理 yylex的返回值， 需要对yylex的返回值进行约定

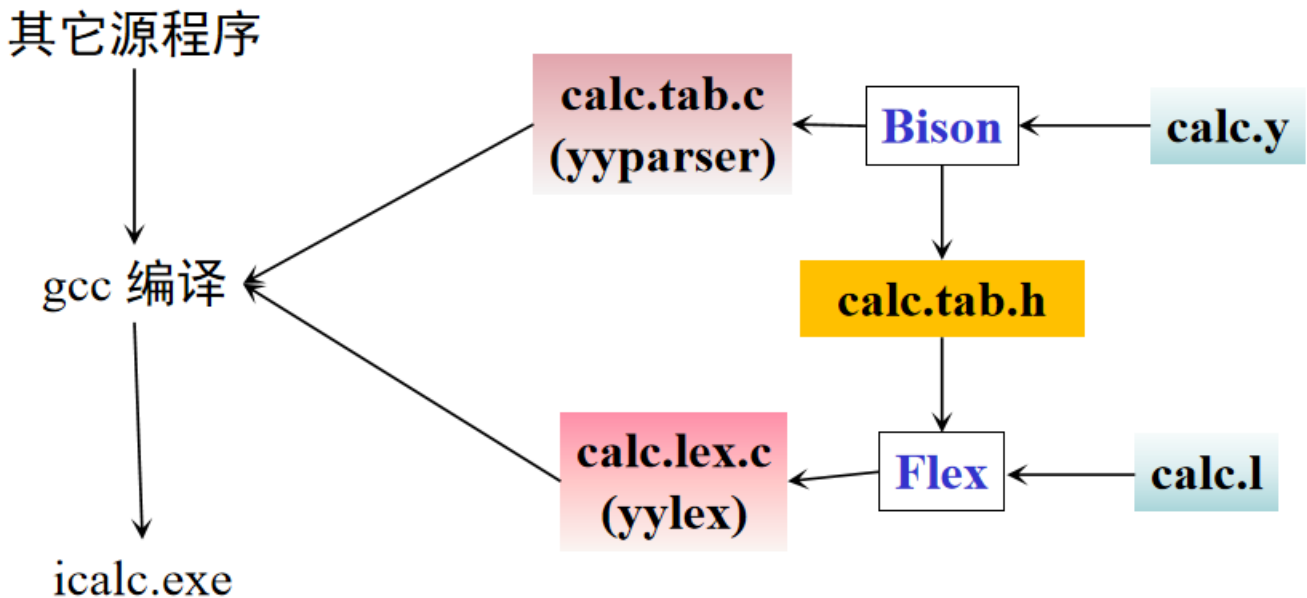
② 在Bison源文件中定义 %token NUMBER

③ 生成语法分析器时使用 -d 参数生成 xxx.tab.h

即 bison -d xxx.y 得到 xxx.tab.h 和 xxx.tab.c

其中 xxx.tab.h中有 #define NUMBER 258，该头文件需要被包含进 xxx.l 的定义部分

关于Bison/Yacc




```
bison -d -o calc.tab.c calc.y          // 生成 calc.tab.h, calc.tab.c
flex -o calc.lex.c calc.l              // 生成 calc.lex.c, calc.l中要include "calc.tab.h"
gcc –o icalc.exe calc.lex.c calc.tab.c -lm   // 编译、链接
```

关于Bison/Yacc

① yylex的返回的是词法记号（单词编号）,而一词法记号有相应的属性：标识符、常数等

② 词法记号的属性值如何返回？语法分析器如何访问 token 的属性？

③ 约定使用全局变量 yylval, yylval 的类型默认为整数类型，用户可自己定义, 一般定义为 union 类型

Bison 文件的结构

... 定义 ...

%%

... 语法规则（文法及对应的处理） ...

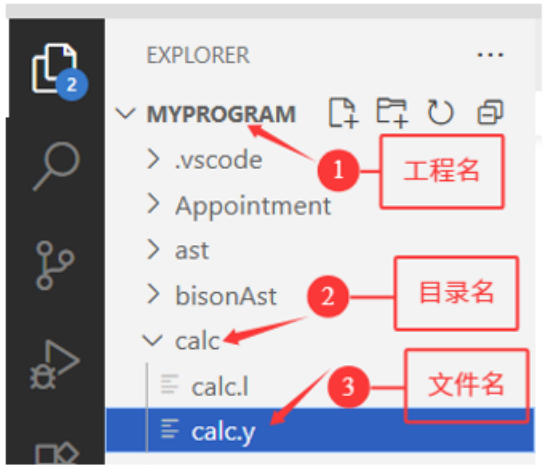
%%

... 主函数定义 ...

其中： 定义部分的C源代码需要放在 %{ 与 %} 之间

以简易计算器为例：编写 calc.y 语法分析

```
// 语法分析器 Bison 文件 calc.y
%{
    #include <stdio.h>
    #include <stdlib.h>
    extern int yyparse(void);
    extern int yylex(void);
    extern void yyerror(char *s);
%}
```



calc > calc.y

```
1  // Bison calc.y
2
3  %{
4  #include <stdio.h>
5  #include <stdlib.h>
6  extern int yyparse(void);
7  extern int yylex(void);
8  extern void yyerror(char *s);
9  %}
```

编写 calc.y 语法分析(续)

```
//声明double_value的类型
%union {
    double double_value;
}
// 声明非终结符的类型
%type <double_value> expression
%type <double_value> term
%type <double_value> factor
%type <void> statement // statement 不需要返回值
```

```
10
11  //声明double_value的类型
12  %union {
13      double double_value;
14  }
15  // 声明非终结符的类型
16  %type <double_value> expression
17  %type <double_value> term
18  %type <double_value> factor
19  %type <void> statement // statement 不需要返回值
```

编写 calc.y 语法分析(续)

//声明终结符

%token <double_value> NUMBER

%token ADD SUB MUL DIV

%token LPAREN RPAREN SEMICOLON ABS EOL

//定义优先级和结合性

%left ADD SUB

%left MUL DIV

```
20
21    //声明终结符
22    %token <double_value> NUMBER
23    %token ADD SUB MUL DIV
24    %token LPAREN RPAREN SEMICOLON ABS EOL
25
26    //定义优先级和结合性
27    %left ADD SUB
28    %left MUL DIV
29
```

编写 calc.y 语法分析(续)

```
//定义产生式文法的语法规则
%%
    input: | input statement
           ;
    statement:
          expression EOL { printf("=:%g\n", $1);}
          ;
    expression: expression ADD term   { $$ = $1 + $3;}
          | expression SUB term       { $$ = $1 - $3; }
          | term                      { $$ = $1; }
          ;
    term: term MUL factor   { $$ = $1 * $3; }
       | term DIV factor    { $$ = $1 / $3; }
       | factor             { $$ = $1; }
       ;
    factor: NUMBER                     { $$ = $1; }
         | LPAREN expression RPAREN    { $$ = $2; }
         | SUB factor                  { $$ = -$2; }
         | ADD factor                  { $$ = $2; }
         ;
%%
```

```
30  %%
31  input:
32      | input statement
33      ;
34  statement:
35          expression EOL { printf("=:%g\n", $1);}
36          ;
37  expression: expression ADD term   { $$ = $1 + $3;}
38            | expression SUB term   { $$ = $1 - $3; }
39            | term                  { $$ = $1; }
40            ;
41
42  term: term MUL factor   { $$ = $1 * $3; }
43      | term DIV factor   { $$ = $1 / $3; }
44      | factor            { $$ = $1; }
45      ;
46
47  factor: NUMBER          { $$ = $1; }
48        | LPAREN expression RPAREN { $$ = $2; }
49        | SUB factor       { $$ = -$2; }
50        | ADD factor       { $$ = $2; }
51        ;
52
53  %%
```

编写 calc.y 语法分析(续)

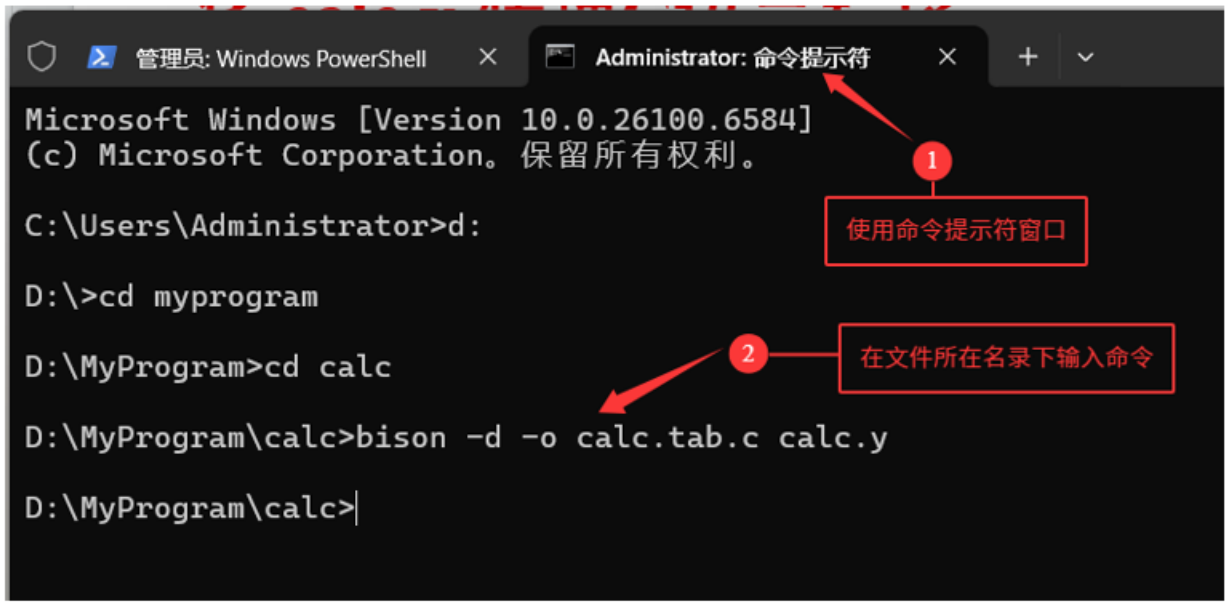
```
//定义C语言的主函数
int main() {
    printf("Enter an expression or press Ctrl+C to exit (e.g., 2 + 3 * 4): \n");
    while (1) {
        yyparse();
    }
    return 0;
}
```

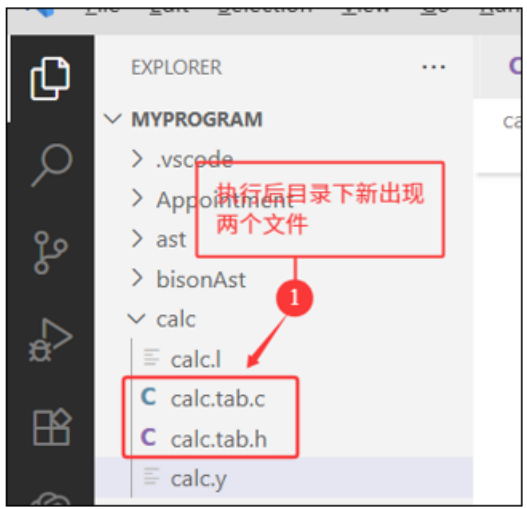
```
54
55  int main() {
56      printf("Enter an expression or press Ctrl+C to exit (e.g., 2 + 3 * 4): \n");
57      while (1) {
58          yyparse();
59      }
60      return 0;
61  }
```

将 calc.y 转换C语言程序

在微软的命令窗口下，进入到文件所在目录，执行下列命令：

bison -d -o calc.tab.c calc.y



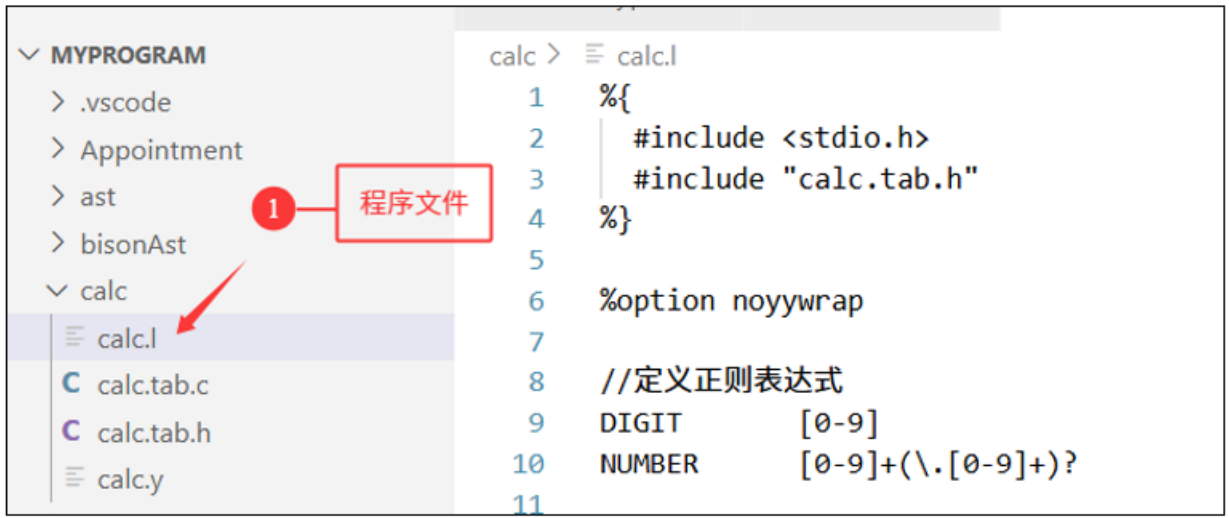


编写 calc.l 词法分析器

```
%{
    #include <stdio.h>
    #include "calc.tab.h"
%}

%option noyywrap

//定义正则表达式
DIGIT     [0-9]
NUMBER    [0-9]+(\.[0-9]+)?
```



编写 calc.l 词法分析器

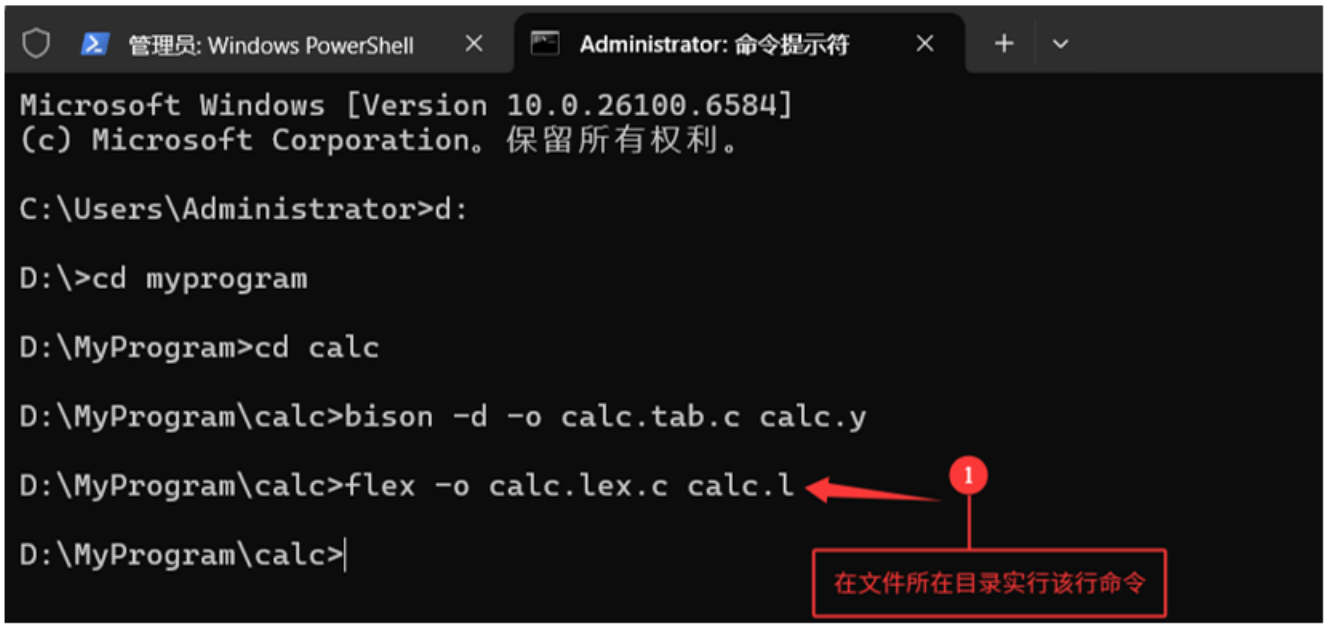
```
%%
    {NUMBER}    { yylval.double_value = atof(yytext); return NUMBER; }
    "+"         { return ADD; }
    "-"         { return SUB; }
    "*"         { return MUL; }
    "/"         { return DIV; }
    "("         { return LPAREN; }
    ")"         { return RPAREN; }
    ";"         { return SEMICOLON; }
    "|"         { return ABS; }
    \n          { return EOL;}
    [\t]        ; // 忽略空格、制表符和换行符
%%
void yyerror(const char *s) {
    fprintf(stderr, "%s\n",s);
    return;
}
```

```
13
14      {NUMBER}    { yylval.double_value = atof(yytext); return NUMBER; }
15      "+"         { return ADD; }
16      "-"         { return SUB; }
17      "*"         { return MUL; }
18      "/"         { return DIV; }
19      "("         { return LPAREN; }
20      ")"         { return RPAREN; }
21      ";"         { return SEMICOLON; }
22      "|"         { return ABS; }
23      \n          { return EOL;}
24      [\t]        ; // 忽略空格、制表符和换行符
25
26  %%
27
28  void yyerror(const char *s) {
29      fprintf(stderr, "%s\n",s);
30      return;
31  }
```

编写 calc.l 词法分析器

执行下列命令生成词法分析器的C语言程序

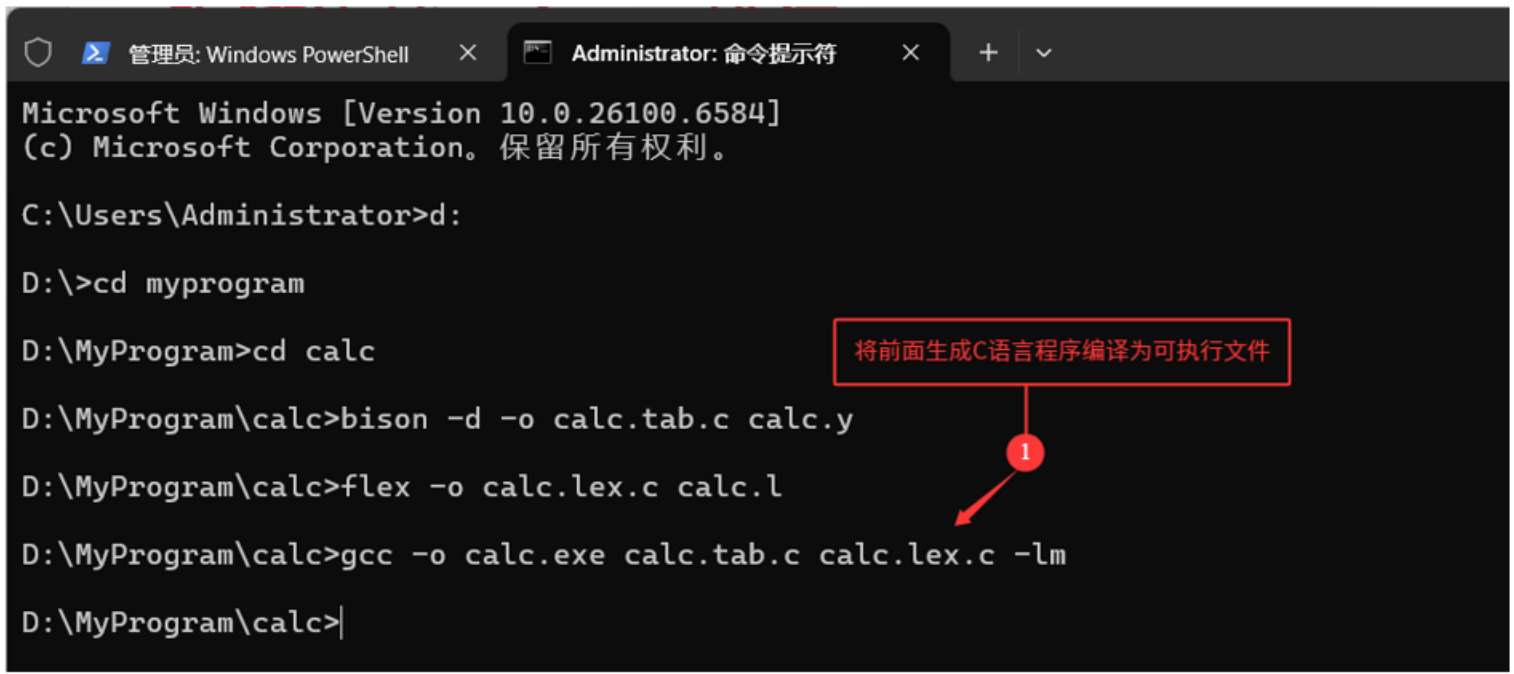
flex -o calc.lex.c calc.l



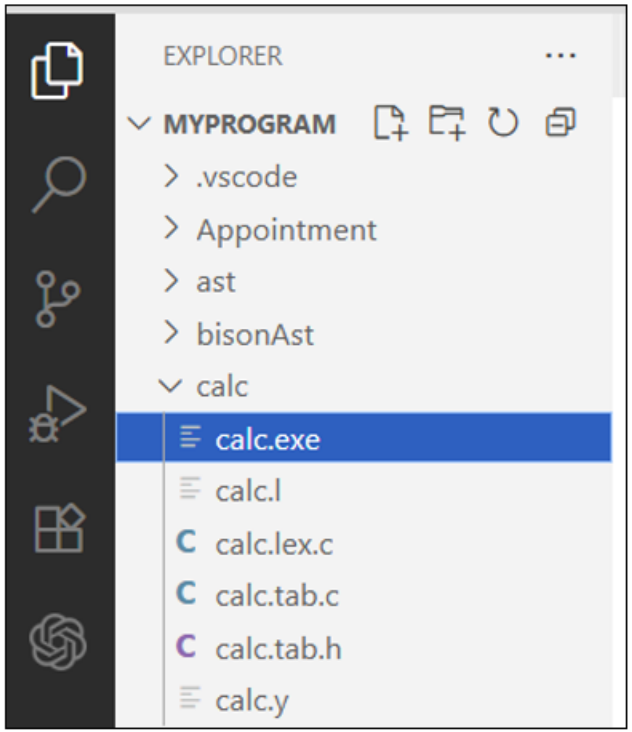
编译生成 calc.exe可执行文件

执行下列命令编译C语言程序为执行文件

gcc -o calc.exe calc.tab.c calc.lex.c -lm



执行 calc.exe文件




```
管理员: Windows PowerShell    Administrator: 命令提示符 - ca
(c) Microsoft Corporation。保留所有权利。

C:\Users\Administrator>d:

D:\>cd myprogram

D:\MyProgram>cd calc

D:\MyProgram\calc>bison -d -o calc.tab.c calc.y

D:\MyProgram\calc>flex -o calc.lex.c calc.l

D:\MyProgram\calc>gcc -o calc.exe calc.tab.c calc.lex.c -lm

D:\MyProgram\calc>calc
Enter an expression or press Ctrl+C to exit (e.g., 2 + 3 * 4):
34*(89-56)/4+90
=:370.5
23+52*(40-21)
=:1011
```

LR 分析生成抽象语法树

在Bison文件的定义区

```
%union {
        int     iValue;
        past    pAst;
};

%token <iValue> NUMBER
%type  <pAst>   program expr factor term
```

其中%union定义了yylval的类型

终结符：%token <iValue> NUMBER

指定文法中NUMBER为整型，通过 yylval.iValue访问

非终结符：%type <pAst> program expr factor term

指定program、expr等为past类型，通过yylval.pAst访问

LR 分析生成抽象语法树

```
expr: factor
      | expr '+' factor      {$$ = newExpr('+', $1, $3);}
      | expr '-' factor      {$$ = newExpr('-', $1, $3);}
      ;
factor: term
      | factor '*' term      {$$ = newExpr('*', $1, $3);}
      | factor '/' term      {$$ = newExpr('/', $1, $3);}
      ;
term: NUMBER                 {$$ = newNum($1);}
      | '-' term             {$$ = newExpr('-', NULL, $3);}
      ;
```

注意：newExpr函数需要在头文件中声明，C文件中实现

实验任务

编写SysY语言所提供文法的LR语法分析程序

(1)编写生成“语法树”的相关程序，包括

bison源程序 lrparser.y

flex源程序 lrlex.l

语法树相关程序 ast.h 和 ast.c

(2)其它相关函数(如main函数)等 main.c,

(3)使编译得到的rdparser最终从命令行读取要分析的程序test.c,分析后调用showAst打印该程序的结构。

实验安排要求

1. 所有文件都以 utf-8 进行统一编码保存！！！

   (检查环境为Linux，如出现乱码等错误，将会扣分！)

2. 提交文件：

   ① flex源程序 lrlex.l

   ② bison源程序 lrparser.y

   ③ 生成语法树代码 ast.c 及头文件 ast.h

   ④ 其它相关代码 main.c

   ⑤ 实验报告 LR语法分析.docx